



A markdown-it plugin that adds an `id` attribute to headings and optionally permalinks.

English | [中文 \(v7.0.1\)](#)

Overview

This plugin adds an id attribute to headings, e.g. `## Foo` becomes `<h2 id="foo">Foo</h2>`.

Optionally it can also include [permalinks](#), e.g. `<h2 id="foo"><a class="header-anchor" href="#foo">Foo</a></h2>` and a bunch of other variants!

- **Usage**
- User-friendly URLs
- Manually setting the id attribute
- Compatible table of contents plugin
- Parsing headings from HTML blocks
- Browser example
- **Permalinks**
 - Header link

- [Link after header](#)
- [Link inside header](#)
- [ARIA hidden](#)
- [Custom permalink](#)
- [Debugging](#)
- [Development](#)

Usage

```
const md = require('markdown-it')
  .use(require('markdown-it-anchor'),
    opts)
```

Name	Description	Default
tabIndex	Value of the tabIndex attribute on headings, set to false to disable.	-1
uniqueSlugStartIndex	Index to start with making duplicate	1

Name	Description Default
getTokensText	Called with token and info after rendering. A custom function to get the text contents of the title from its tokens.
	7
	2

Name	Description Default
slugify	A function to render permalinks, see permalinks below.
custom slugification function.	A custom slugification function. See index.js
callback	undefined

Name	Description Default
level	Minimum level to apply anchors, or 1 array of selected levels.
permalink	undefined

Name	Description	Default
slugs	unique.	

All headers greater than the minimum level will have an `id` attribute with a slug of their content. For example, you can set `level` to 2 to add anchors to all headers but `h1`. You can also pass an array of header levels to apply the anchor, like `[2, 3]` to have an anchor on only level 2 and 3 headers.

If a permalink renderer is given, it will be called for each matching header to add a permalink. See [permalinks](#) below.

If a `slugify` function is given, you can decide how to transform a heading text to a URL slug. See [user-friendly URLs](#).

The callback option is a function that will be called at the end of rendering with the token and an info object. The info object has title and slug properties with the token content and the slug used for the identifier.

We set by default `tabindex="-1"` on headers. This marks the headers as focusable elements that are not reachable by keyboard navigation. The effect is that screen readers will read the title content when it's being jumped to. Outside of screen readers, the experience is the same as not setting that attribute. You can override this behavior with the `tabIndex` option. Set it to `false` to remove the attribute altogether, otherwise the value will be used as attribute value.

Finally, you can customize how the title text is extracted from the markdown-it tokens (to later generate the slug). See [user-friendly URLs](#).

User-friendly URLs

Starting from v5.0.0, markdown-it-anchor dropped the [string](#) package to retain our core value of being an impartial and secure library. Nevertheless, users looking for backward compatibility may want the old slugify function:

```

        .filter(token => !['html_inline',
            'image'].includes(token.type))
        .map(t => t.content)
        .join('')
    }

    const md = require('markdown-it')()
    .use(require('markdown-it-anchor'),
        { getTokensText })

```

Customizing the slugify input

Additionally, if you want to further customize the title that gets passed to the `slugify` function, you can do so by customizing the `getTokensText` function, that gets the plain text from a list of markdown-it inline tokens:

```
function getTokensText (tokens) {  
  return tokens  
}
```

15

10

Customizing the slingshot input

```

const slugify =
  require('@sindresorhus/
    slugify')(
    const md = require('markdown-it')().
      use(require('markdown-it-anchor'),
        { slugify(s) =<= s }
      )
    )

```

```

mpm install string
const string = require('string')
const slugify = s =>
  string(s).slugify().toString()

const md = require('markdown-it')()
  .use(require('markdown-it-anchor'), { slugify })

```

sensible approach for the vast majority of use cases. An alternative approach is to include every token’s content except for <code>html_inline</code> and <code>image</code> tokens, which yields the exact same results as the previous approach with a stock <code>markdown-it</code>, but would also include custom tokens added by any of your <code>markdown-it</code> plugins, which might or might not be desirable for you. Now you have the option!	Slugifying with state If you need access to the <code>markdown-it</code> state from the <code>slugify</code> function, e.g. to access <code>state.env</code>, you can use <code>slugifyWithState</code> instead. <pre>const md = require('markdown-it')() .use(require('markdown-it-anchor'), {</pre>	<pre>slugifyWithState: (title, state) => `\${state.env.id}-\${   slugify(title)}` })</pre> Manually setting the id attribute You might want to explicitly set the <code>id</code> attribute of your headings from the <code>Markdown</code> document, for example to keep them consistent across translations. <code>markdown-it-anchor</code> is designed to reuse any existing <code>id</code>, making markdown-it-attrs a	perfect fit for this use case. Make sure to load it before <code>markdown-it-anchor</code>! Then you can do something like this: <pre># Your title {#your-custom-id}</pre> The anchor link will reuse the <code>id</code> that you explicitly defined. Compatible table of contents plugin Looking for an automatic table of contents (TOC) generator? Take a look at markdown-it-																																	
17	18	19	20																																	
24	23	22	21																																	
by <code>markdown-it-anchor</code>: heading_close tokens that can be handled sequence of <code>heading_open</code>, <code>inline</code>, and <code>transform the html_block_tokens</code> into a with their <code>marky-markdown</code> parser, and not designed to, you can do like npm does <code>markdown-it-anchor</code> for this even though it's That said if you really want to use give you a solid base. handling duplicated slugs and IDs, this should While this still needs extra work like	<pre>h.innerHTML = `<a href="#\${slug}">   {h.innerHTML}</a>   }   console.log(root.toString())   Or with a (not accessible) GitHub-style anchor, replace the h.innerHTML part with:   h.insertAdjacentHTML('afterbegin',     ` `)</pre>	<pre>const { parse } = require('node-html- parser') const root = parse(html) for (const h of root.querySelectorAll('h1, h2, h3, h4, h5, h6')) { const slug = h.getAttribute('id') slugify(h.textContent) h.setAttribute('id', slug) }</pre>	toc-done-right it's made from the ground to be a great companion of this plugin. Parsing headings from HTML blocks <code>markdown-it-anchor</code> doesn't parse HTML blocks, so headings defined in HTML blocks will be ignored. If you need to add anchors to both HTML headings and <code>Markdown</code> headings, the easiest way would be to do it on the final HTML rather than during the <code>Markdown</code> parsing phase:																																	
<pre>const md = require('markdown-it')() .use(require('@npmcorp/marky- markdown/lib/plugin/html- heading')) .use(require('markdown-it-anchor'), opts)</pre> While they use regexes to parse the HTML and it won't gracefully handle any arbitrary HTML, it should work okay for the happy path, which might be good enough for you. You might also want to check this implementation which uses Cheerio for a	more solid parsing, including support for HTML attributes. The only edge cases I see it failing with are multiple headings defined in the same HTML block with arbitrary content between them, or headings where the opening and closing tag are defined in separate <code>html_block</code> tokens, both which should very rarely happen. If you need a bulletproof implementation, I would recommend the first HTML parser approach I documented instead.	Browser example See example.html. Permalinks Version 8.0.0 completely reworked the way permalinks work in order to offer more accessible options out of the box. You can also make your own permalink. Instead of a single default way of rendering permalinks (which used to have a poor UX on	screen readers), we now have multiple styles of permalinks for you to chose from. <pre>const anchor = require('markdown-it- anchor') const md = require('markdown-it')() md.use(anchor, { permalink: anchor.permalink[styleOfPermalink] (permalinkOpts) })</pre>																																	
25	26	27	28																																	
32	31	30	29																																	
(though you could add it with CSS using <code>:before</code> if you like). It's so simple it doesn't have any behaviour to custom, and it's also accessible out of the box without any further configuration, hence it doesn't have other options than the common ones described above. You can find this style on the MDN as well as HTTP Archive and their Web Almanac, which to me is a good sign that this is a thoughtful way of implementing permalinks.	Header link This style wraps the header itself in an anchor link. It doesn't use the symbol option as there's no symbol needed in the markup	<table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td>The symbol in the permalink</td><td>A custom anchor.</td><td></td></tr><tr><td>renderHref</td><td>A custom permalink href</td><td>See</td></tr><tr><td>rendering function.</td><td>A custom rendering function.</td><td></td></tr><tr><td>permalink.js</td><td>A custom permalink</td><td>See</td></tr><tr><td>renderAttrs</td><td>A custom attributes</td><td>See</td></tr><tr><td>permalink.js</td><td></td><td></td></tr><tr><td>permalink.js</td><td></td><td></td></tr></table>	Name	Description	Default	The symbol in the permalink	A custom anchor.		renderHref	A custom permalink href	See	rendering function.	A custom rendering function.		permalink.js	A custom permalink	See	renderAttrs	A custom attributes	See	permalink.js			permalink.js			<table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td>The class of the header-permalink anchor</td><td>permalink anchor</td><td>#</td></tr><tr><td>symbol</td><td></td><td></td></tr></table>	Name	Description	Default	The class of the header-permalink anchor	permalink anchor	#	symbol		
Name	Description	Default																																		
The symbol in the permalink	A custom anchor.																																			
renderHref	A custom permalink href	See																																		
rendering function.	A custom rendering function.																																			
permalink.js	A custom permalink	See																																		
renderAttrs	A custom attributes	See																																		
permalink.js																																				
permalink.js																																				
Name	Description	Default																																		
The class of the header-permalink anchor	permalink anchor	#																																		
symbol																																				

This is also the style that I chose for my own blog. <table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td>safariReaderFix</td><td>Add a span inside the link so Safari shows headings in reader view.</td><td>false (for backwards compatibility)</td></tr></table> 33	Name	Description	Default	safariReaderFix	Add a span inside the link so Safari shows headings in reader view.	false (for backwards compatibility)	<table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td></td><td>See common options.</td><td></td></tr></table> <pre>const anchor = require('markdown-it-anchor') const md = require('markdown-it')() md.use(anchor, {</pre> 34	Name	Description	Default		See common options .		<pre>permalink: - e.anchor.permalink.headerLink() }) <h2 id="title">Title</h2></pre> The main caveat of this approach is that you can't include links inside headers. If you do, consider the other styles. Also note that this pattern breaks reader mode in Safari, an issue you can also notice 35	on the referenced websites above. This was already reported to Apple but their bug tracker is not public. In the meantime, a fix mentioned in the article above is to insert a span inside the link. You can use the safariReaderFix option to enable it. <pre>const anchor = require('markdown-it-anchor') const md = require('markdown-it')() md.use(anchor, {</pre> 36																					
Name	Description	Default																																		
safariReaderFix	Add a span inside the link so Safari shows headings in reader view.	false (for backwards compatibility)																																		
Name	Description	Default																																		
	See common options .																																			
40 styles label and aria- assistive returns the hidden visually- the title and that takes A function required for undefined, labelledby. or aria- Description Default	39 describedby aria- aria-label, hidden, visually- visually- one of style of link, The (sub) Description Default Name leverages a visuallyHiddenClass to hide the assistive text from the visual experience.	38 hidden to not pollute the experience, and text. It makes the permalink symbol aria- different methods of customizing the assistive and adding the permalink <i>after</i> it, giving you It works by leaving the header itself alone, header link. style gives you much more freedom than the reader experience of your permalinks, this If you want to customize further the screen Link after header	37 <pre>permalink: anchor.permalink.headerLink({ safariReaderFix: true }) }) <h2 id="title">Title</h2></pre>																																	
<table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td>visuallyHiddenClass</td><td>The class you use to make an element visually hidden.</td><td>undefined, required for visually-hidden style</td></tr><tr><td>space</td><td>Add a space between the assistive text and the</td><td>true</td></tr></table> 41	Name	Description	Default	visuallyHiddenClass	The class you use to make an element visually hidden.	undefined, required for visually-hidden style	space	Add a space between the assistive text and the	true	<table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td></td><td>permalink symbol.</td><td></td></tr><tr><td></td><td>Placement of the permalink symbol relative to the assistive text, can be before or</td><td></td></tr><tr><td>placement</td><td></td><td>after</td></tr></table> 42	Name	Description	Default		permalink symbol.			Placement of the permalink symbol relative to the assistive text, can be before or		placement		after	<table><tr><th>Name</th><th>Description</th><th>Default</th></tr><tr><td></td><td>after the header.</td><td></td></tr><tr><td>wrapper</td><td>Opening and closing wrapper string, e.g. [' ', ' '].</td><td>null</td></tr><tr><td></td><td>See common options.</td><td></td></tr></table> 43	Name	Description	Default		after the header.		wrapper	Opening and closing wrapper string, e.g. [' ', ' '].	null		See common options .		<pre>const anchor = require('markdown-it-anchor') const md = require('markdown-it')() md.use(anchor, { permalink: anchor.permalink.linkAfterHeader({ style: 'visually-hidden', assistiveText: title => `Permalink to "\${title}"`, visuallyHiddenClass: 'visually-hidden',</pre> 44
Name	Description	Default																																		
visuallyHiddenClass	The class you use to make an element visually hidden.	undefined, required for visually-hidden style																																		
space	Add a space between the assistive text and the	true																																		
Name	Description	Default																																		
	permalink symbol.																																			
	Placement of the permalink symbol relative to the assistive text, can be before or																																			
placement		after																																		
Name	Description	Default																																		
	after the header.																																			
wrapper	Opening and closing wrapper string, e.g. [' ', ' '].	null																																		
	See common options .																																			
48 aria-describedby and aria-labelledby variants This removes the need to customize the assistive text to your locale and doesn't need <pre><h2 id="title">Title</h2> </pre>	47 <pre>const anchor = require('markdown-it-anchor') const md = require('markdown-it')() md.use(anchor, { permalink: anchor.permalink.linkAfterHeader({ style: 'aria-label', assistiveText: title => `Permalink to "\${title}"`</pre> experience when using a screen reader through a translation service.	46 By using a visually hidden element for the assistive text, we make sure that the assistive text can be picked up by translation services, as most of the popular translation services (including Google Translate) currently ignore aria-label. If you prefer an alternative method for the assistive text, see other styles: aria-label variant This removes the need from a visually hidden span, but will likely hurt the permalink	45 <pre><h2 id="title">Title</h2> Permalink to "Title" </pre> <pre>wrapper: [' ', ' ']</pre>																																	

a visually hidden span either, but since the anchor will be described by just the text of the title without any context, it might be confusing. <pre>const anchor = require('markdown-it-anchor') const md = require('markdown-it')() md.use(anchor, { permalink: anchor.permalink.linkAfterHeader({</pre>	<pre>style: 'aria-describedby' // Or `aria-labelledby` }) }) <h2 id="title">Title</h2> #</pre>	Link inside header This is the equivalent of the default permalink in previous versions. The reason it's not the first one in the list is because this method has accessibility issues. If you use a symbol like just # without adding any markup around, screen readers will read it as part of every heading (in the case of #, it could be read “pound”, “number” or “number sign”) meaning that if you title is “my beautiful title”, it will read “number sign	my beautiful title” for example. For other common symbols,  is usually read as “link symbol” and ¶ as “pilcrow”. Additionally, screen readers users commonly request the list of all links in the page, so they'll be flooded with “number sign, number sign, number sign” for each of your headings. I would highly recommend using one of the markups above which have a better experience, but if you really want to use this markup, make sure to pass accessible HTML
49	50	51	52
<pre><a> # Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump to heading Jump</pre>			

Development # Build the library in the `dist/` directory. npm run build # Watch file changes to update `dist/`. npm run dev # Run tests, will use the build version so make sure to build after 65	# making changes. npm test 66	67	68
72	71	70	69
73	74	75	76
88	79	78	77